Sky Custom Reports: Transition and Improvement Proposal

Prepared for	Sky-Tech
Prepared by	Charan Tej Kandukuri / ValueLabs
Date	November 4, 2026
Version	0.1 (Draft)
Subject	Takeover of the `sky-custom-reports` service and proposed approach

Contents

1. Basis of This Document
2. Our Understanding of the Current System
3. Immediate Action Items (from the Handover)
4. Long-Term Direction Recommended by the Previous Owner
5. Responses to the Client's Questions
6. Beyond Technology: Process and Product Ideas (Out-of-the-Box Ideas)

Additional sections will follow in later versions of this document.

1. 📄 Basis of This Document

The observations, solutions and tactics in this document are based, **to the best of our knowledge**, on the handover document provided by Sergei Privalov (*Sky Custom Reports: Handover Document, Draft v1*) and on the points discussed during the client call.

We thank Sky-Tech for sharing the handover. It gives a clear overview of the service's architecture, its history and its known challenges. As the handover is a first draft, some areas are naturally described at a high level or not yet covered, for example:

- the exact data held at each pipeline stage (e.g. section snapshots, relevance decisions),
- how documents are prepared and indexed before reaching this service,
- the measured limits of the system for large cases.

A **more in-depth understanding of the current implementation** would allow us to propose a more robust and better-targeted solution. Access to the source code, a technical walkthrough with the previous owner or the owners of related services, and access to the relevant environments would help most. With that understanding we can fully answer the questions raised by the client:

1. What is the available context: how much can the system process and hold today?
2. What is the right user-facing solution?
3. What would the best possible build look like with today's technology?
4. How can users get the best output from the system as it exists now?

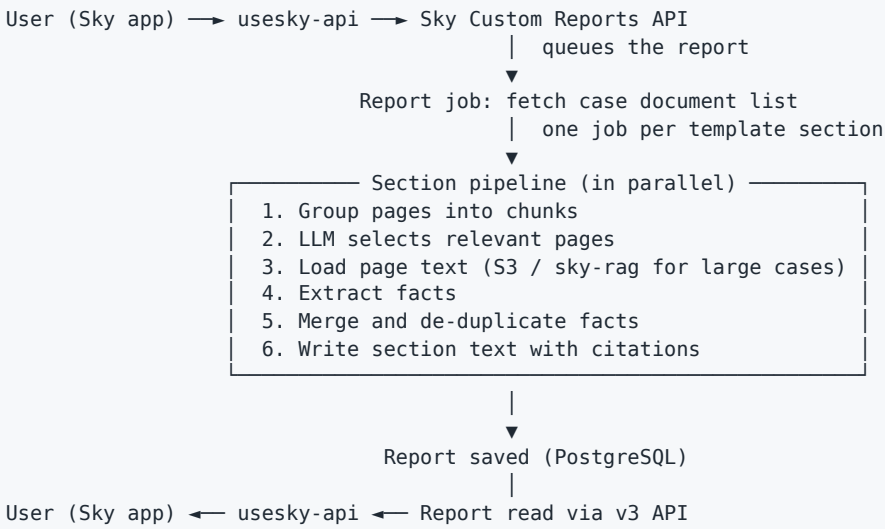
Statements in this document should therefore be read as our current assessment, to be confirmed and refined as more detail becomes available.

---

2. 🔍 Our Understanding of the Current System

Based on the handover, `sky-custom-reports` is a microservice that generates template-based reports from a case's documents using large language models (LLMs). Google Gemini is the primary model and Anthropic Claude is the fallback; a separate Canadian instance uses Vertex AI for data-residency reasons. The service's only client is `usesky-api`, the main Sky backend. A user starts report generation for a case in the Sky application, and the request passes through `usesky-api` to this service. The service fetches the case's document list, splits the work by **template section**, and generates the sections **in parallel** (a template may have around 50 sections). The finished report is read back through the v3 API.

Each section is produced by a multi-stage LLM pipeline that works from the **actual page content** of the documents, not from summaries, so that every fact can be cited back to its source page. Pages are grouped into chunks, an LLM identifies which pages are relevant to the section, the page text is loaded (from S3, and for large cases filtered through the `sky-rag` search service), facts are extracted and then merged and de-duplicated, and finally the section text is written with citations. The handover notes that the pipeline has been validated up to roughly 1,000 pages, and that the hardest open problem is achieving exhaustive fact extraction, readable text and correct citations at the same time.



> **Not clear from the handover:** how the case document list is produced, and how documents are converted into page text before they reach this service. It is also not stated what information the relevance step uses to decide whether a page is relevant. We would confirm these points with the team.

---

3. ✅ Immediate Action Items (from the Handover)

The handover lists the following open work items, which the previous owner recommends completing first. We propose to take them on as the initial part of the transition.

#	Action item	Description	Reference
1	**Enable cross-section deduplication in production**	Turn on the `CROSS_SECTION_DEDUP_ENABLED` flag in production and add it to the infrastructure configuration. The feature is built but defaults to *off* in code. It reduces facts repeated across report sections.	Feature flag
2	**Commit and stabilise the report-gold evaluation**	Commit the evaluation work that hasn't been committed yet on branch `devin/sky-3944-report-gold` , and make the evaluation run reproducible.	SKY-3944
3	**Complete "Stage 6" of stuck-processing remediation**	Add a heartbeat and an orphan reconciler for **section** jobs, following the existing pattern used for templates, so failed jobs are detected reliably.	`docs/stuck-processing-remediation-plan.md`
4	**Template-section identity round-trip**	The backend is ready; this is waiting on the frontend template editor.	`docs/preserve-templatesection-identity-plan.md`

#	Action item	Description	Reference
5	**Fix or remove a dead script**	`eval:report-fidelity` in `workers/package.json` points to a file that doesn't exist. The working runner is in the root `scripts/` folder.	`workers/package.json`
6	**Update the README**	It describes the old queue scheme, omits the chronology and human-input section types, and doesn't reflect the current v3/v4 APIs (v1/v2 are deprecated but still live).	`README`
7	**Complete the Grafana dashboard**	Several monitoring panels were built for the previous pipeline and need updating for the current one.	Grafana
8	**Resolve a possible duplicate constants file**	`constants.ts` and `constants/reports.constants.ts` in `api/src/reports` may duplicate each other.	`api/src/reports`

> 📌 **Note:** item 1 also addresses the client's concern about duplicated information in reports. Items 2 and 7 support the client's request for measurable quality.

---

4. 🧭 Long-Term Direction Recommended by the Previous Owner

The handover also sets out the previous owner's strategic recommendations, described there as the author's opinion. We summarise them below as the starting point for our long-term proposal.

4.1 Strategic recommendations

#	Recommendation	Rationale given in the handover
1	**Migrate queues from BullMQ (Redis) to AWS SQS**	SQS is already used by other Sky services. The current parent/child job model is not ideal, and SQS would simplify reliability and the stuck-job problem.
2	**Cache shared LLM context during report preparation**	Avoid re-sending identical context to the LLM, reducing cost.
3	**Revisit the use of Genkit**	Workers show slow memory growth that appears linked to Genkit, and newer agentic approaches are now available.
4	**Move prompt configuration from S3 into code or environment variables**	Prompt configuration is loaded only at worker startup, so the current setup suggests "hot" updates that don't actually happen.
5	**Add a final whole-report pass**	Sections are generated in parallel, and no step currently checks the report as a whole for consistency across sections.
6	**Validate scaling to 5,000–10,000 pages**	The pipeline is validated to roughly 1,000 pages; behaviour and cost at larger volumes are unproven.

#	Recommendation	Rationale given in the handover
7	**Roll out cross-section deduplication**	Already built; needs enabling in the infrastructure (see Section 3, item 1).
8	**Make facts a persistent data layer**	The main long-term direction, described below.

4.2 Main long-term direction: facts as a persistent data layer

Current situation. Facts are extracted from the documents on **every** report generation and exist only during that single run.

Proposed direction (as described in the handover):

Step	Description
1	**Extract** facts during pipeline stages, with structure and metadata: source, page, date and type (e.g. medical, chronology, insurance event).
2	**Persist** the facts in the database.
3	**Deduplicate** facts across generation runs and across report sections.
4	**Retrieve** facts per user query, using semantic or graph search and topic clustering (e.g. medical, chronology).
5	**Assemble** a coherent report from the retrieved facts.

Expected benefits noted in the handover:

- It fits naturally with **agentic approaches**, where an agent selects relevant groups of facts and composes the report, instead of following a rigid linear pipeline.
- It addresses the **core quality challenge** of the system: extracting facts exhaustively while keeping the prose readable and the citations correct.
- It makes **repeat generations cheaper**, since facts don't need to be extracted again each time.

> 💡 **Our view:** this direction matches the client's priorities: fewer missing facts, less duplication, correct ordering of events, and the ability to improve a report from about 75% to 95% without rerunning it. We will build on it in the proposed solution sections of this document.

---

5. 💬 Responses to the Client's Questions

This section responds to the four questions raised during the client call. For each question we separate **what the handover tells us** from **our proposed approach**. As noted in Section 1, these responses reflect our current understanding and would be refined after a deeper review of the implementation.

Summary

#	Question	Our response in brief
1	What is the available context?	The real limit is the **system's measured capacity** (pages per case and facts per section at an acceptable completeness and cost), not the model's published context size. We propose to measure it with a structured benchmark.
2	What is the user-facing solution?	A **review-and-correct workflow**: source-linked text, targeted corrections at section (and later fact) level, and single-section regeneration, supported by simpler template creation.
3	What would the best build look like today?	Build on the previous owner's direction: a **persistent, cited fact store** with consolidation (deduplication, consistent naming, conflict detection), reports composed from facts, human review, and continuous quality evaluation.
4	How can users get the best output now?	Enable existing capabilities (cross-section deduplication), **measure quality** with the evaluation work already started, and use **targeted edits** instead of full reruns.

5.1 What is the available context?

What the handover tells us.

- The system never sends a whole case to the LLM at once. Pages are grouped into chunks, relevant pages are selected, facts are extracted per chunk, merged, and then used to write each section.
- The pipeline has been validated up to roughly **1,000 pages**. Cases of **5,000–10,000 pages are not yet validated**.
- Gemini is the primary model, Claude the fallback, and Vertex AI is used for the Canadian instance.

Our view. The question is less about any model's published context size, which is public and changes with each new model, and more about how much **this system** can reliably handle. That depends on:

- the pipeline's own limits (chunking, relevance selection, merging, and how many facts the final writing step can hold),
- the quality of each model on long inputs, including the fallback model,
- cost at report scale (around 50 sections per report, plus retries and reruns).

These limits can only be established by measurement.

Our proposed approach.

Step	Description
1	Build a **benchmark** of real, de-identified cases of increasing size (e.g. 100, 500, 1,000 and 5,000 pages) with human-verified reference reports.
2	Measure **fact recall** (share of expected facts that appear in the report), along with time and cost, at each size.
3	Add **per-stage fact counts** (pages relevant → facts extracted → facts after merge → facts in the final text) to show *where* facts are lost.
4	Publish a clear **capacity statement**, e.g. "cases up to *N* pages: *X*% of key facts captured, at about *$Y* per report".

> **Not clear from the handover:** how many facts a single section can hold before quality drops, and at what page count the `sky-rag` page filtering takes over. The benchmark above is designed to answer both.

5.2 What is the user-facing solution?

What the handover and the client call tell us.

- The main users are organisations that serve physicians (for example IME and medical-assessment companies) and act as the *human in the loop*. Physicians mainly review and sign off the final report.
- Reports often reach about **75%** of the required quality. Reaching **90–95%** currently means rerunning generation, which consumes tokens and often needs technical support.
- Creating a template for each report type is a significant effort for clients.
- Existing capabilities include **AI-edit** for sections and reports, **human-input** sections, section **previews**, and **template generation from a sample PDF**. Preserving template-section identity is ready in the backend and waiting on the frontend editor.

Our proposed approach.

Area	Proposal
Review with sources	Every statement links to its source page, so reviewers can verify content quickly. *(Citations already exist.)*
Targeted corrections	Correct one section with AI-edit or regenerate a single section, instead of rerunning the whole report. Once facts are persisted (Section 4.2), corrections can be made at **fact level** and **kept across reruns**.
Reporting gaps	A reviewer can flag missing information ("the 2023 MRI is missing"), which triggers a targeted re-check of the relevant pages rather than a full regeneration.
Simpler templates	Start from a sample report (existing template generation), refine in the template editor, and reuse common sections across report types.

Structured input. During the call, the client asked what users should provide as input so that, even with large organisation-wide material, outputs stay predictable and flaws can be traced to their cause, for example to a context-window limit. We see this mainly as a question of **input structure**, which goes beyond formatting:

Aspect	Meaning	Example
Labelled	Every input states what it is	Case document or organisation policy; document type and date
Scoped	Only what is needed is supplied	The policies that apply to this report type, not the organisation's full document set
Consistent	The same input structure is used every time for a report type	Every IME report receives the same kinds of inputs, so runs can be compared
Measured	The amount of input used is recorded	How much context each section actually used

With structured and measured inputs, the pattern of errors points to the likely cause:

What is observed	Likely cause
Errors increase as input size increases	Context-window limit
Errors occur only with certain document types	Parsing or extraction of those documents
Errors occur at every input size	Prompt or template issue
Errors occur only when certain policies are included	Conflicting or unclear organisation documents

5.3 What would the best build look like with today's technology?

What the handover tells us. The previous owner recommends making **facts a persistent data layer** that is extracted once, stored with source metadata, deduplicated, and used to compose reports, possibly through agentic approaches (Section 4.2).

Our proposed approach. We would build on that direction, in phases, without a full rewrite:

```
Case documents
     │
     ▼
1. Extract facts once per case (typed, dated, cited to page)
     │
     ▼
2. Fact store ── consolidate: remove duplicates, unify names,
     │            flag contradictions, track page coverage
     ▼
3. Compose each section from its facts (events ordered by date,
     │                                  citations by fact reference)
     ▼
4. Human review ── corrections saved back to the fact store
     │
     ▼
5. Continuous quality evaluation (recall, accuracy, citations)
```

How this addresses the client's concerns:

Client concern	How the proposed build addresses it
Missing critical facts	Every page is tracked as processed or not, so gaps are visible. Fact recall is measured on every release.
Duplicate information	Duplicates are merged once in the fact store, keeping all their sources, so every section sees a single copy.
Order of events	Facts carry normalised dates, and chronologies are ordered by date before the text is written.
Contradictions	Conflicting facts (e.g. two different dates for the same event) are flagged and shown to the reviewer, not silently chosen.
Consistent naming	One shared list of names and terms (people, providers, facilities) is applied to every section.
Cost and reruns	Facts are reused across reruns and new templates; corrections don't require full regeneration.
Reliability	Retry handling is simplified and job processing is made durable, building on the previous owner's recommendations (e.g. moving to SQS).

Agentic composition, where an agent queries the fact store and fills gaps, would be introduced where it adds clear value, with limits on steps and cost so that results stay predictable.

5.4 How can users get the best output from the current system now?

These steps work within the current system, before larger changes are made:

#	Action	Benefit
1	**Enable cross-section deduplication** in production (Section 3, item 1)	Fewer repeated facts across sections
2	**Complete the report-gold evaluation** (Section 3, item 2) and add a **fact recall** measure	Quality becomes measurable and comparable between releases
3	**Add per-stage fact counts** to the existing metrics	Shows where facts are being lost
4	**Use targeted edits** (AI-edit, single-section regeneration, human-input sections) instead of full reruns	Faster and cheaper path from 75% to 95%
5	**Validate templates with previews** before generating full reports	Problems with section instructions are caught early, at lower cost
6	**Update the monitoring dashboard** for the current pipeline (Section 3, item 7)	Faster diagnosis of failed or poor-quality generations

> 💡 **Our view:** steps 1–3 can begin immediately and give a measured baseline. That baseline will guide which parts of the longer-term build (Section 5.3) to prioritise.

---

6. 🧩 Beyond Technology: Process and Product Ideas (Out-of-the-Box Ideas)

Many of the challenges in report generation are not purely technical. In our view, they come from asking the system to produce a **finished report in a single step**, with people correcting it afterwards. The ideas below change **what is delivered**, **when people are involved** and **how quality is defined**. They complement the technical direction in Sections 4 and 5, and can largely be introduced step by step.

6.1 Deliver the evidence first, then the report

- **A verified case file before the report.** Before any report is written, deliver a **timeline of events** and an **index of key evidence**: a short list of the important facts, each with the document and page where it was found. The reviewer approves this list, and the report is then written from the approved version.
- **The chronology as the backbone.** Most medical summaries and assessment reports rest on "what happened, and when". When the timeline is correct and approved, duplicates, ordering errors and contradictions are largely resolved before the report is written.

Example of an index of key evidence:

#	Key fact	Source
1	Injured at work on 2 March 2024	Incident report, page 1
2	MRI showed a disc herniation at L4–L5	MRI report, page 14
3	Prescribed ibuprofen 400 mg	Clinic note, page 3
4	Referred to physiotherapy	Physician's letter, page 2
5	Returned to light duties on 10 June 2024	Employer letter, page 1

A reviewer can check a list like this in minutes, missing items are easy to notice, and a correction made once applies everywhere the fact is used.

6.2 Involve people early and briefly, not late and at length

- **A short case brief before generation.** The reviewer confirms a few key points: date of injury, main conditions, key providers, the most important documents, and the question the report must answer. Clear direction up front avoids much of the later correction work.

  The brief is **prepared by the system, not typed by the reviewer**. The system makes a quick first pass over the case documents and proposes each key point with its source. The reviewer then confirms, corrects, removes or adds items, and generation starts once the brief is confirmed. The question the report must answer is usually already stated in the instruction or referral letter, so little typing is needed.

  Example of a case brief presented for confirmation:

```
CASE BRIEF — please confirm                                  Source
──────────────────────────────────────────────────────────────────────
Date of injury        2 March 2024                [✓] [✎]  Incident report p.1
Main conditions       Lower back injury (L4–L5)   [✓] [✎]  MRI report p.14
                      Left shoulder strain        [✓] [✎]  Clinic note p.3
Key providers         Dr A. Patel (GP)            [✓] [✎]  Clinic note p.1
                      City Physio Clinic          [✓] [✎]  Referral letter p.2
Most important docs   MRI report, Specialist letter  [✓] [✎]
Report must answer    Is the claimant fit to return  [✓] [✎]  Instruction letter p.1
                      to full duties?
                                              [+ Add item]
──────────────────────────────────────────────────────────────────────
                                        [ Confirm and generate ]
```

  [✓] confirm, [✎] edit. With around 5–10 items, the brief takes a few minutes. Errors such as a wrong date of injury are corrected once, before generation, instead of in several sections of the finished report. For simple cases, the brief could be optional.

- **Focus review on what carries risk.** Rather than rereading the whole report, reviewers are shown conflicts between sources, critical facts supported by only one source, and uncertain items.
- **Show what was not used.** Each report lists pages reviewed but not cited, pages that couldn't be read, and documents not referenced. Possible omissions become visible instead of silent.

6.3 Improve how records arrive

- **Records intake standards.** Clients supply records sorted, labelled and with an index of what's included. Alternatively, records preparation could be offered as a service. This reduces duplicates and makes problems easier to trace, in line with the client's point about structured input (Section 5.2).
- **Process records as they arrive.** Cases often build up over weeks. Building and reviewing the timeline as documents arrive means most of the work is already checked by the time a report is needed.

6.4 Define quality together with the client

- **Agreed acceptance criteria per report type.** For each report type, agree a short **must-have checklist** of facts and elements with the client. Quality then means "meets the agreed checklist", an objective and shared measure.
- **An audit summary with every report.** A one-page summary of documents reviewed, coverage, conflicts found and how they were resolved, and who approved the report. In insurance and legal settings this adds defensibility.

6.5 Simplify template creation

- **Templates from examples.** Clients provide a few past reports they were satisfied with, and the structure and style are taken from those, so no template has to be built from scratch.
- **A library of standard report types.** Many assessment reports follow common formats. Ready-made standard templates that clients adjust would reduce the effort for each new client.

6.6 Make every correction count

- **Corrections become organisation rules.** When reviewers repeatedly make the same correction (for example "always include medication dosage", or a preferred way of referring to the person), it becomes a standing rule for that organisation. The service improves for each client the longer they use it.

6.7 Clarify the division of work

- **The system organises, people judge.** The system handles the high-volume work: reading, extracting, organising and cross-referencing. People keep the judgement: what matters for the medical opinion and how conflicts are resolved.
- **An optional managed review service.** For clients who prefer not to perform the review themselves, a review service could be offered as an additional tier.

6.8 How these ideas address the client's concerns

Client concern	Ideas that address it
Missing critical facts	Case brief, list of unused material, agreed checklist
Duplicate information	Records intake standards, chronology as the backbone
Order of events	Chronology approved first
Contradictions	Risk-focused review, audit summary

Client concern	Ideas that address it
Difficulty of quality assurance	Agreed checklist, audit summary
Improving from 75% to 95%	Evidence first, case brief, corrections become rules
Effort to create templates	Templates from examples, standard template library
Very large inputs	Intake standards, processing records as they arrive, case brief
Physicians' limited time	Evidence-first review, audit summary, managed review service

> 💡 **Our view:** rather than asking "how can the system write a perfect report?", we suggest asking **"how can the right evidence be confirmed early, so the report becomes largely a formality?"** These ideas would be refined with the client and introduced alongside the technical improvements.
